

Assignment 4: Fingerprint Quality Assessment & Scoring Pipeline

YellowSense Technologies — Technical Assessment Report | Sairam

Overview

I built the 5-metric quality gate as specified -- blur, brightness, glare, ROI completeness, and ridge clarity, combined into a 0-100 composite score. To actually test it I captured 20 photos of my own finger with my phone (5 good, 5 blurry, 5 dark, 5 with glare) and ran them through the pipeline. The first version worked fine on the good/blurry/dark cases, but I hit a real problem with glare detection that took some debugging to sort out -- details below.

1. What threshold did you set for blur? How did you decide?

I kept the Laplacian variance threshold at 10.0. When I ran it against my real photos the separation was pretty clean on its own -- my 5 good shots scored between 11.65 and 17.59, and the 5 blurry ones scored 1.58 to 4.01, so there was a wide enough gap that I didn't feel the need to move the threshold. What I didn't expect was that 3 of my 5 "dark" photos also scored under 10 (5.05-8.90), not because my hand was shaking but just because low light kills contrast, so the Laplacian variance drops too. That meant the app was telling me "too blurry" on photos that were actually just dark. I fixed it by checking brightness before blur in the guidance logic, so the message reflects what's actually wrong.

2. Which metric was hardest to implement correctly? What went wrong first?

Glare, easily. My first version flagged glare by counting pixels brighter than 240 -- straight out of the spec. I tested it on my 5 glare photos (taken by shining a torch on my finger) and not a single one got flagged. Zero glare_fraction across the board. I wrote a small script to print out the max pixel value in each of those photos and found they topped out around 177-220, never anywhere near 240. Turned out my phone's auto-exposure was compensating for the bright spot before the sensor ever saturated, so a fixed cutoff was never going to catch it. I switched to flagging pixels that are unusually bright relative to that specific photo's own average (mean + 1.5 standard deviations) instead of a fixed number, since a glare spot is really about being brighter than its surroundings, not hitting some absolute value. After that fix, 4 of my 5 glare photos got correctly flagged. One interesting side effect: 2 of my blurry photos also picked up some glare_fraction (0.09-0.14) with the new method, because motion blur smears bright spots across nearby pixels in a way that looks similar to a glare spike. It didn't change the final result since blur gets checked first, but it was a good reminder that these metrics aren't as independent as they look on paper.

3. What is NFIQ2? Why is a score designed for contact scanners not reliable for phone camera images?

NFIQ2 is NIST's standard 0-100 quality score for fingerprints, but it was built for flatbed contact scanners -- fixed 500 DPI, even lighting, consistent black-and-white ridge contrast. None of that holds for a phone camera. The distance you hold your phone changes the effective DPI, the finger is curved so you get perspective distortion a scanner never sees, and you're dealing with actual skin tone, shadows, and reflections instead of a clean scan. So NFIQ2 tends to score phone captures low even when they're genuinely fine to use, because it's

measuring against conditions that don't apply here -- which is really the whole reason this assignment exists, to build something calibrated for the actual capture conditions instead.

4. Name 3 other quality problems you'd add checks for in a real deployment.

First, angle/perspective -- if the finger's tilted relative to the camera, the ridge spacing gets warped and that'll mess with matching later even if the image looks fine at a glance. Second, having more than one finger in frame -- easy to do by accident with a contactless capture, and it would throw off segmentation. Third, distance -- if the finger's too far away you lose effective resolution, and if it's too close you go outside the camera's focal range and get blur for a totally different reason than shaky hands.

5. If a rural agricultural worker's fingerprints are naturally worn and give consistently poor ridge clarity scores, what should the system do differently for them?

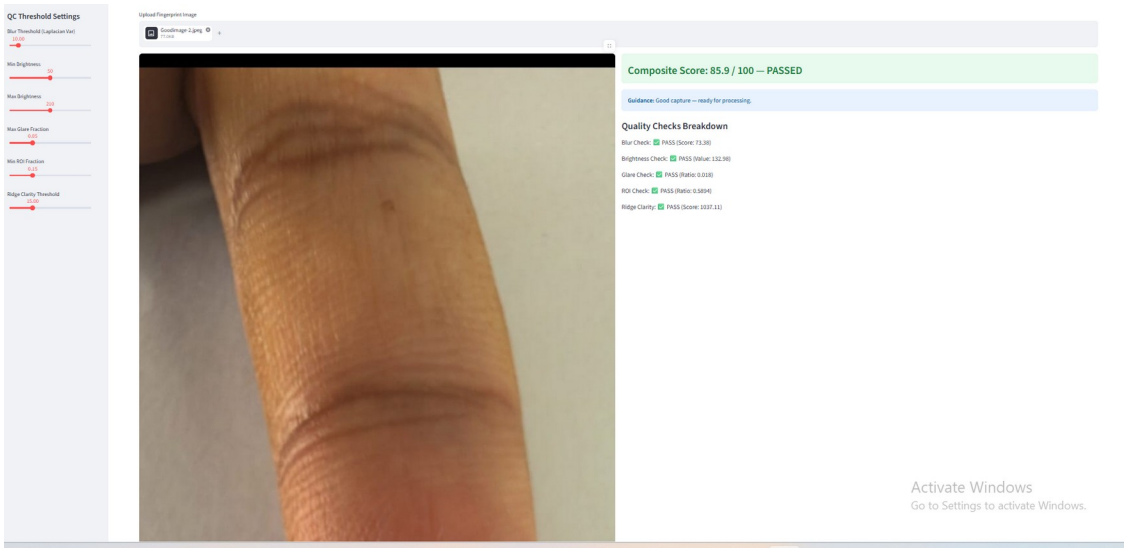
A single fixed ridge-clarity threshold would basically lock these users out, which isn't acceptable. I'd handle it a few ways: let the threshold adapt per-user based on what was captured at enrollment rather than one global number for everyone; run something like CLAHE to boost local contrast before scoring, since worn ridges often still have real structure, just lower contrast; give people the option to try a different finger that might be less worn; and if none of that works, fall back to a different biometric like face or iris rather than just rejecting the person outright.

Test Results (20 real phone photos)

Category	Result	Notes
Good (5)	5/5 passed	Composite scores 70.7-99.1
Blurry (5)	5/5 correctly rejected	Blur scores 1.58-4.01 vs. good's 11.65-17.59
Dark (5)	5/5 correctly rejected	Brightness 9.88-24.68, all below the 50 minimum
Glare (5)	4/5 correctly rejected	1 false negative -- see note below

One of my glare photos (Glareimage-3.jpeg) actually passed, with a glare_fraction of just 0.0003. I don't think that's a bug -- I probably held the torch at a slightly different angle or distance on that particular shot, so it just didn't produce much of a hotspot, even though I'd filed it under "glare" when I took it. So the algorithm's call was arguably right, my labeling was just optimistic. Overall 19 of 20 photos (95%) landed in the category I expected.

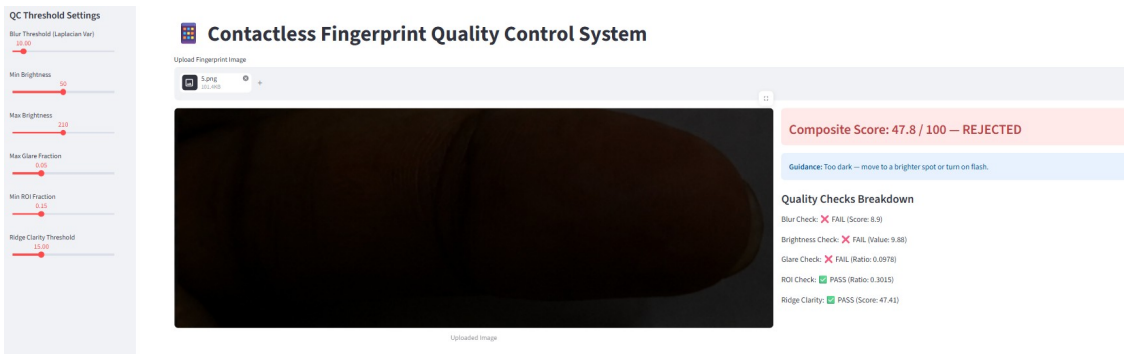
Streamlit Dashboard Screenshots



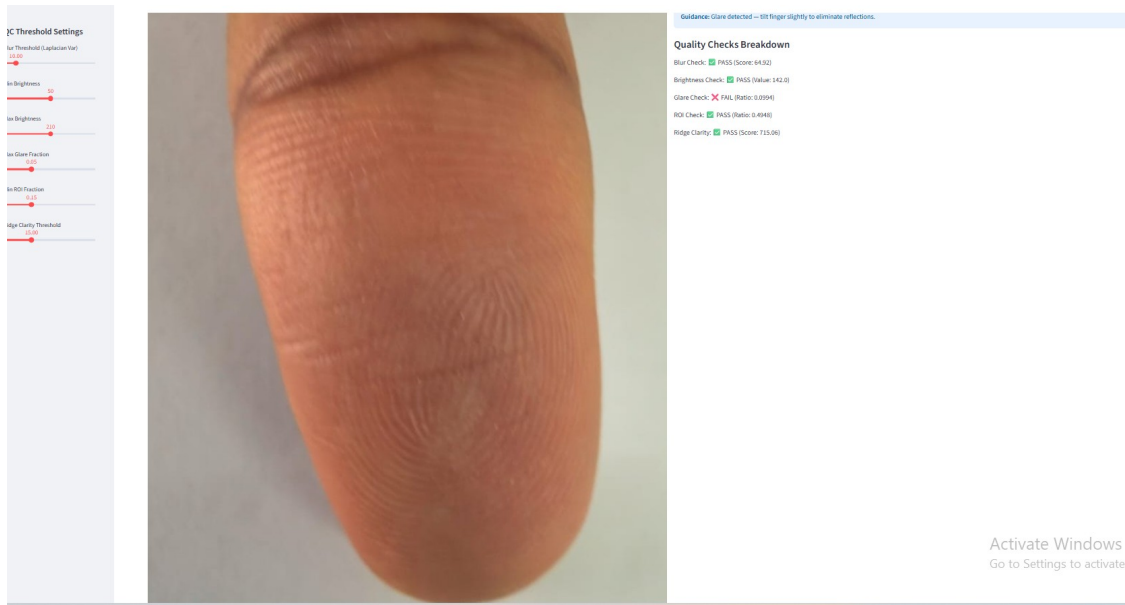
Good capture -- PASSED (Composite 85.9/100)



Blurry capture -- REJECTED (Composite 74.0/100)

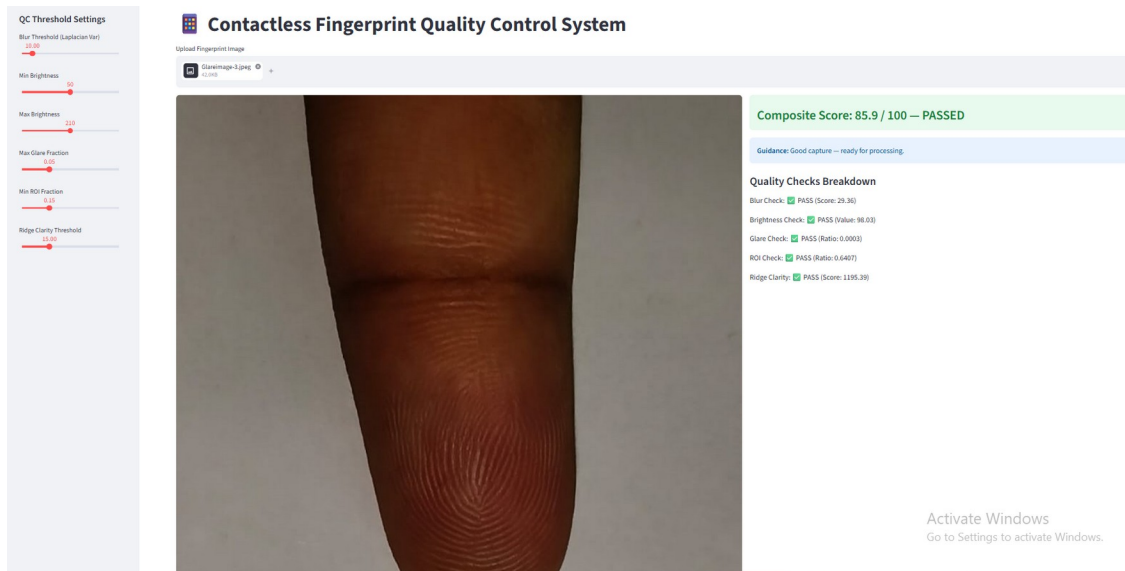


Dark capture -- REJECTED (Composite 47.8/100)



Glare capture -- REJECTED (glare ratio 0.0994)

Bonus: the glare false-negative case (Glareimage-3.jpeg) discussed above, for reference:



Glare-condition photo that PASSED (glare ratio 0.0003) -- see Q2 discussion.